



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 16: Clean Code (Cap. 5)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

2 de Octubre, 2025

Cuestionario cap 7 y 8

- **[Cap 7]** Una ventaja de usar excepciones es que permiten separar la lógica de negocios de la lógica para manejar errores.

- **[Cap 7]** Una ventaja de usar excepciones es que permiten separar la lógica de negocios de la lógica para manejar errores.
- **[Cap 7]** Generalmente no vale la pena usar "checked exceptions"

- **[Cap 7]** Una ventaja de usar excepciones es que permiten separar la lógica de negocios de la lógica para manejar errores.
- **[Cap 7]** Generalmente no vale la pena usar "checked exceptions"
- **[Cap 7]** Es mejor usar excepciones que retornar códigos de error.

- **[Cap 7]** Una ventaja de usar excepciones es que permiten separar la lógica de negocios de la lógica para manejar errores.
- **[Cap 7]** Generalmente no vale la pena usar "checked exceptions"
- **[Cap 7]** Es mejor usar excepciones que retornar códigos de error.
- **[Cap 7]** El SPECIAL CASE PATTERN nos permite crear nuevos tipos de excepciones.

- **[Cap 8]** Del capítulo se infiere que es mejor no usar librerías externas pues sus APIs (y cambios) no están bajo tu control.

- **[Cap 8]** Del capítulo se infiere que es mejor no usar librerías externas pues sus APIs (y cambios) no están bajo tu control.
- **[Cap 8]** Según el capítulo deberías considerar crear una clase "Team", que contenga la lista de unidades de un equipo, en tu proyecto.

- **[Cap 8]** Del capítulo se infiere que es mejor no usar librerías externas pues sus APIs (y cambios) no están bajo tu control.
- **[Cap 8]** Según el capítulo deberías considerar crear una clase "Team", que contenga la lista de unidades de un equipo, en tu proyecto.
- **[Cap 8]** Una ventaja de los learning tests es que sirven como boundary tests.

- **[Cap 8]** Del capítulo se infiere que es mejor no usar librerías externas pues sus APIs (y cambios) no están bajo tu control.
- **[Cap 8]** Según el capítulo deberías considerar crear una clase "Team", que contenga la lista de unidades de un equipo, en tu proyecto.
- **[Cap 8]** Una ventaja de los learning tests es que sirven como boundary tests.
- **[Cap 8]** Al dividir el trabajo en un equipo de desarrollo es posible que tu módulo dependa de una API que aún no existe. En esa situación no tiene mucho sentido que avances en tu módulo pues todo lo que hagas terminará cambiando una vez que te entreguen la API con la que debes trabajar.

Formatting



“When people look under the hood, we want them to be impressed with the neatness, consistency, and attention to detail that they perceive. We want them to be struck by the orderliness. We want their eyebrows to rise as they scroll through the modules. We want them to perceive that professionals have been at work.”

– Robert C. Martin.

Formatting

Todo equipo de desarrollo cuenta con una serie de reglas que buscan que su código se vea “como si hubiese sido escrito por la misma persona”. Para ello cuentan con guías de estilo.

Formatting

Todo equipo de desarrollo cuenta con una serie de reglas que buscan que su código se vea “como si hubiese sido escrito por la misma persona”. Para ello cuentan con guías de estilo.

Por ejemplo, Google tiene guías de estilo para:

- C#
- Python
- JavaScript
- ... y varios lenguajes más.

Formatting

Todo equipo de desarrollo cuenta con una serie de reglas que buscan que su código se vea “como si hubiese sido escrito por la misma persona”. Para ello cuentan con guías de estilo.

Por ejemplo, Google tiene guías de estilo para:

- C#
- Python
- JavaScript
- ... y varios lenguajes más.

Todo buen software necesita que sus archivos tengan un estilo consistente, el lector debe ser capaz de confiar que las reglas que lee en un archivo sean las mismas que se siguen en otros archivos.

Formatting

Todo equipo de desarrollo cuenta con una serie de reglas que buscan que su código se vea “como si hubiese sido escrito por la misma persona”. Para ello cuentan con guías de estilo.

Por ejemplo, Google tiene guías de estilo para:

- C#
- Python
- JavaScript
- ... y varios lenguajes más.

Todo buen software necesita que sus archivos tengan un estilo consistente, el lector debe ser capaz de confiar que las reglas que lee en un archivo sean las mismas que se siguen en otros archivos. Lo último que queremos hacer es añadir más complejidad al código escribiéndolo a tontas y a locas.

Formatting

Todo equipo de desarrollo cuenta con una serie de reglas que buscan que su código se vea “como si hubiese sido escrito por la misma persona”. Para ello cuentan con guías de estilo.

Por ejemplo, Google tiene guías de estilo para:

- C#
- Python
- JavaScript
- ... y varios lenguajes más.

Todo buen software necesita que sus archivos tengan un estilo consistente, el lector debe ser capaz de confiar que las reglas que lee en un archivo sean las mismas que se siguen en otros archivos. Lo último que queremos hacer es añadir más complejidad al código escribiéndolo a tontas y a locas.

¿Que reglas de estilo nos recomienda clean code?

Regla 1: Un archivo debe llamarse igual al único elemento que contiene

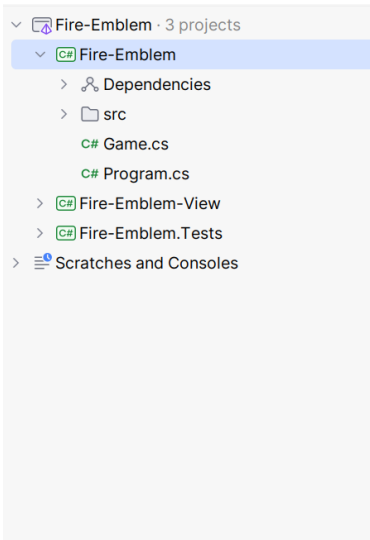
Regla 1: Un archivo debe llamarse igual al único elemento que contiene

Tener más de un elemento en un archivo hace más difícil encontrar lo que estoy buscando

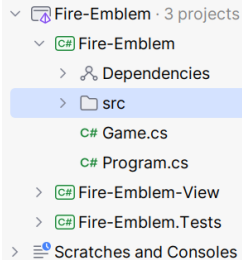
Regla 1: Un archivo debe llamarse igual al único elemento que contiene

```
2 public interface Effect
3 {
4     void Apply(GameState state);
5 }
6 public class DamageEffect : Effect
7 {
8     /* Implementation */
9 }
10 public class HealEffect : Effect
11 {
12     /* Implementation */
13 }
14 public class SummonEffect : Effect
15 {
16     /* Implementation */
17 }
```

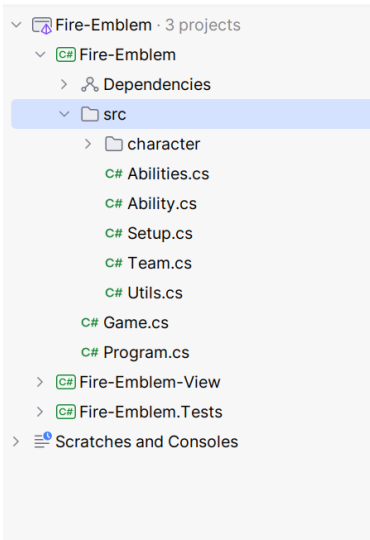
¿Dónde se encuentra la clase `LunaEffect`?



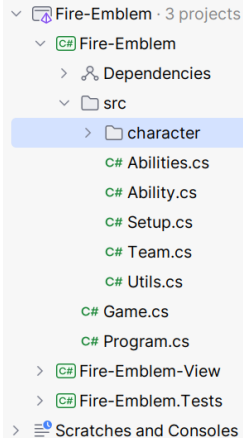
¿Dónde se encuentra la clase LunaEffect?



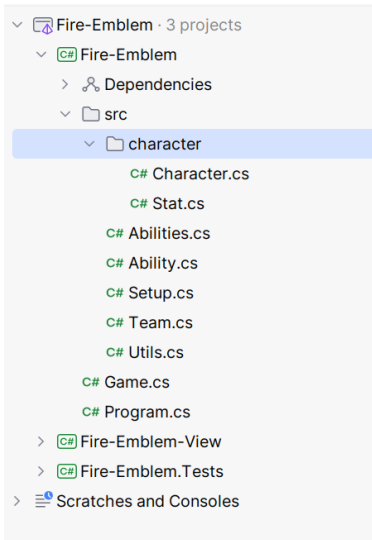
¿Dónde se encuentra la clase LunaEffect?



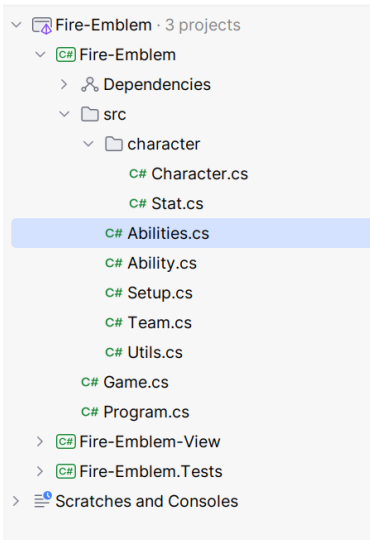
¿Dónde se encuentra la clase LunaEffect?



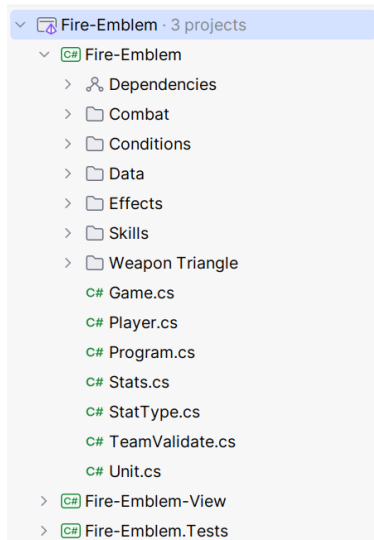
¿Dónde se encuentra la clase LunaEffect?



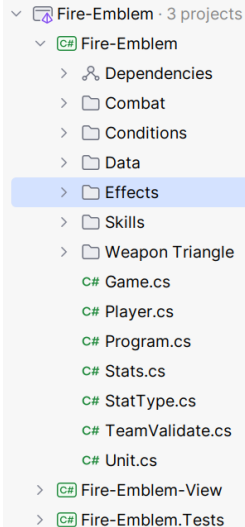
¿Dónde se encuentra la clase LunaEffect?



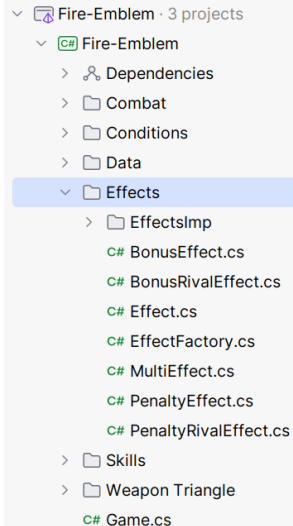
¿Dónde se encuentra la clase LunaEffect?



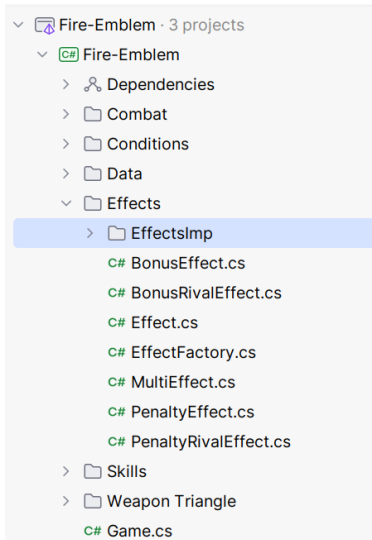
¿Dónde se encuentra la clase LunaEffect?



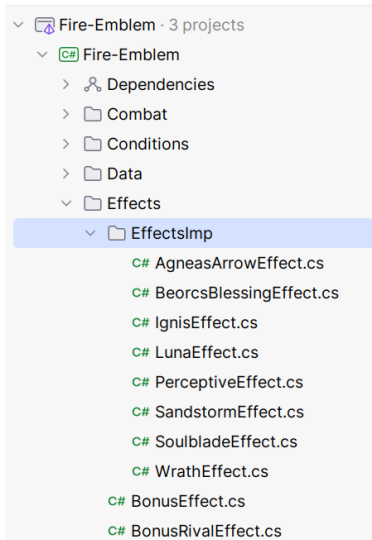
¿Dónde se encuentra la clase LunaEffect?



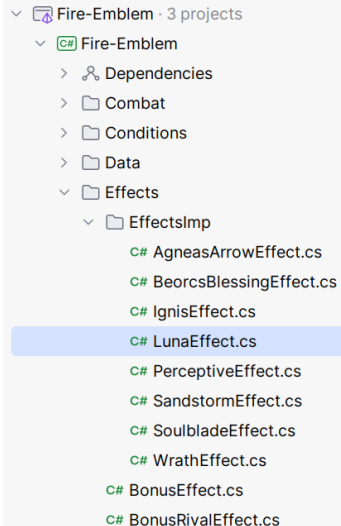
¿Dónde se encuentra la clase LunaEffect?



¿Dónde se encuentra la clase LunaEffect?



¿Dónde se encuentra la clase LunaEffect?



Regla 2: Elementos relacionados de tus clases deben encontrarse juntos en un lugar consistente

Regla 2: Elementos relacionados de tus clases deben encontrarse juntos en un lugar consistente

Los atributos de tu clase deben encontrarse lo más arriba posible, seguido de sus properties, seguido de su constructor y, por último, seguido de sus métodos.

Formatting

Regla 2: Elementos relacionados de tus clases deben encontrarse juntos en un lugar consistente

```
2 public class TurnController {  
3     private readonly UserAction _currentAction;  
4     public void Handle() { /* Implementation */ }  
5     private AffinityCollection GetAffinities() { /* Implementation */ }  
6     private void HandleAffinities(AffinityCollection affinities) { /*  
7         Implementation */ }  
8     private readonly Player _currentPlayer;  
9     private void ClearTargets() { /* Implementation */ }  
10    public TurnController(GameState state, IMegatenView view) { /*  
11        Implementation */ }  
12 }
```

Formatting

Regla 2: Elementos relacionados de tus clases deben encontrarse juntos en un lugar consistente

```
2 public class TurnController {
3     private readonly UserAction _currentAction;
4     private readonly Player _currentPlayer;
5
6     public TurnController(GameState state, IMegatenView view){/*
7     Implementation */}
8
9     public void Handle() {/* Implementation */}
10    private void HandleAffinities(AffinityCollection affinities){/*
11    Implementation */}
12    private AffinityCollection GetAffinities(){/* Implementation */}
13    private void ClearTargets(){/* Implementation */}
14 }
```

Formatting

Regla 2: Elementos relacionados de tus clases deben encontrarse juntos en un lugar consistente

```
2 public class TurnController {
3     private readonly UserAction _currentAction;
4     private readonly Player _currentPlayer;
5
6     public TurnController(GameState state, IMegatenView view){/*
7     Implementation */}
8
9     public void Handle() {/* Implementation */}
10    private void HandleAffinities(AffinityCollection affinities){/*
11    Implementation */}
12    private AffinityCollection GetAffinities(){/* Implementation */}
13    private void ClearTargets(){/* Implementation */}
14 }
```

Lo más importante es que seas consistente

Regla 3: El código debiese contar con una indentación claras y consistentes

Regla 3: El código debiese contar con una indentación claras y consistentes

La indentación nos ayuda muchísimo a la lectura de nuestro código.

```
2 public class FitNesseServer:SocketServer { private FitNesseContext
3 context; public FitNesseServer(FitNesseContext context) {
4 this.context=context;} public void serve(Socket s) {serve(s,10000);}
5 public void serve(Socket s, long requestTimeout){try{FitNesseExpediter
6 sender = new FitNesseExpediter(s, context);
7 sender.setRequestParsingTimeLimit(requestTimeout); sender.start(); }
8 catch(Exception e) { e.printStackTrace(); } } }
```

```
2 public class FitNesseServer : SocketServer {
3     private FitNesseContext _context;
4
5     public FitNesseServer(FitNesseContext context)
6         => _context = context;
7
8     public void serve(Socket s)
9         => serve(s, 10000);
10
11    public void serve(Socket s, long requestTimeout){
12        try{
13            FitNesseExpediter sender = new FitNesseExpediter(s,
14                _context);
15            sender.setRequestParsingTimeLimit(requestTimeout);
16            sender.start();
17        }
18        catch (Exception e){
19            e.PrintStackTrace();
20        }
21    }
```


Regla 4: El código debiese leerse como un diario

Regla 4: El código debiese leerse como un diario

Esto significa lo siguiente para una clase:

- 1 El nombre debe ser simple pero explicativo

Regla 4: El código debiese leerse como un diario

Esto significa lo siguiente para una clase:

- 1 El nombre debe ser simple pero explicativo
- 2 Se debe iniciar con los conceptos de más alto nivel (atributos y funciones publicas)

Regla 4: El código debiese leerse como un diario

Esto significa lo siguiente para una clase:

- 1 El nombre debe ser simple pero explicativo
- 2 Se debe iniciar con los conceptos de más alto nivel (atributos y funciones publicas)
- 3 A medida que bajamos en el archivo, debemos encontrarnos con los detalles

Regla 4: El código debiese leerse como un diario

Esto significa lo siguiente para una clase:

- 1 El nombre debe ser simple pero explicativo
- 2 Se debe iniciar con los conceptos de más alto nivel (atributos y funciones publicas)
- 3 A medida que bajamos en el archivo, debemos encontrarnos con los detalles
- 4 Las variables deben estar declaradas lo más cerca posible a su uso

Regla 4: El código debiese leerse como un diario

Esto significa lo siguiente para una clase:

- 1 El nombre debe ser simple pero explicativo
- 2 Se debe iniciar con los conceptos de más alto nivel (atributos y funciones publicas)
- 3 A medida que bajamos en el archivo, debemos encontrarnos con los detalles
- 4 Las variables deben estar declaradas lo más cerca posible a su uso
- 5 Si una función f llama a una función g , entonces g debe aparecer abajo de f . (Lo más cerca posible)

```
2 public class UserInputProcessor(string filePath){
3     private void SaveToFile(string input)
4         => filePath.AppendAllText(filePath, input);
5
6     private bool ValidateInput(string input)
7         => return !string.IsNullOrEmpty(input) && input.Length >= 3;
8
9     private void TryToSaveToFile(string input){
10         try
11             SaveToFile(input);
12         catch (Exception ex)
13             throw new IOException("Error saving to file.", ex);
14     }
15
16     public void ProcessUserInput(string userInput){
17         if (ValidateInput(userInput))
18             TryToSaveToFile(userInput);
19         else
20             throw new ArgumentException("Invalid input format.");
21     }
22 }
```

```
2 public class UserInputProcessor(string filePath){
3     public void ProcessUserInput(string userInput){
4         if (ValidateInput(userInput))
5             TryToSaveToFile(userInput);
6         else
7             throw new ArgumentException("Invalid input format.");
8     }
9
10    private bool ValidateInput(string input)
11        => return !string.IsNullOrEmpty(input) && input.Length >= 3;
12
13    private void TryToSaveToFile(string input){
14        try
15            SaveToFile(input);
16        catch (Exception ex)
17            throw new IOException("Error saving to file.", ex);
18    }
19
20    private void SaveToFile(string input)
21        => filePath.AppendAllText(filePath, input);
22 }
```


Algunas otras reglas:

- Al igual que con funciones, es preferible que tus archivos de código sean cortos.

Algunas otras reglas:

- Al igual que con funciones, es preferible que tus archivos de código sean cortos.
- Usa saltos de línea para separar grupos de elementos distintos (e.j., métodos).

Algunas otras reglas:

- Al igual que con funciones, es preferible que tus archivos de código sean cortos.
- Usa saltos de línea para separar grupos de elementos distintos (e.j., métodos).
- Tus líneas no deben ser muy largas (menos de 120 caracteres).

Algunas otras reglas:

- Al igual que con funciones, es preferible que tus archivos de código sean cortos.
- Usa saltos de línea para separar grupos de elementos distintos (e.j., métodos).
- Tus líneas no deben ser muy largas (menos de 120 caracteres).
- No mezclen idiomas.

Algunas otras reglas:

- Al igual que con funciones, es preferible que tus archivos de código sean cortos.
- Usa saltos de línea para separar grupos de elementos distintos (e.j., métodos).
- Tus líneas no deben ser muy largas (menos de 120 caracteres).
- No mezclen idiomas.
- Escribe conceptos similares de forma similar y se consistente

**¿Cuáles son los descuentos típicos
en el proyecto debido al cap. 5?**

Descuentos más comunes:

- Tener varias clases/enums/interfaces en el mismo archivo.
- Que el nombre del archivo no sea igual a la clase/enum/interfaz que contiene.
- Toda clase debe comenzar con todos sus atributos, luego sus properties, luego sus constructores y finalmente sus métodos.
- Errores de indentación.
- Orden de los métodos (si `f` llama a `g`, entonces `g` debería estar justo abajo de `f`).
- Spanglish (mejor programen todo en inglés no más).
- Más de 120 caracteres por línea.